Sports Day Website: Project Plan

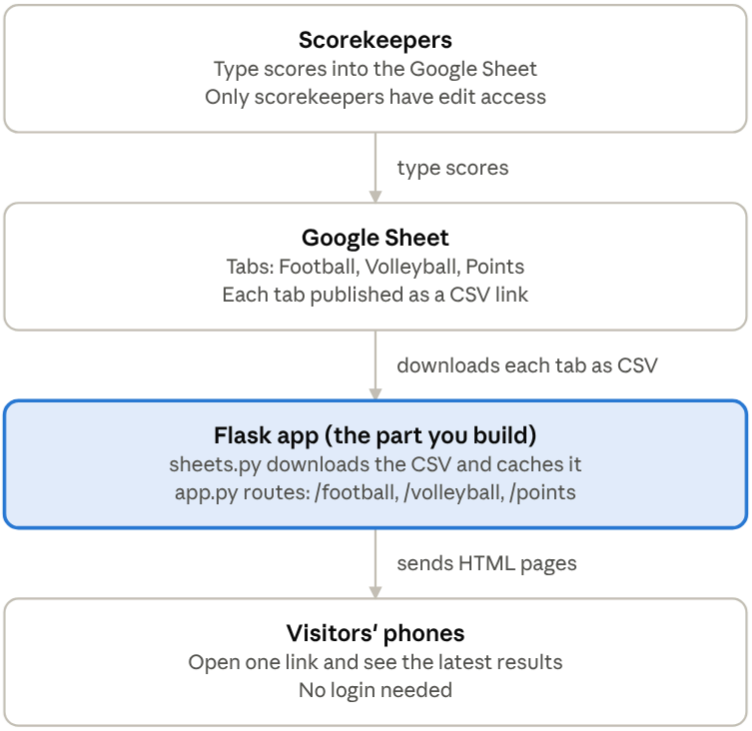
Oct 19, 2026 · @claude

The goal and how it works

The goal is a small website where anyone at sports day opens a link on their phone and sees live football results, volleyball results and the team points table. Scorekeepers never touch the website: they type scores into a Google Sheet, and Flask reads the sheet and shows it.

This keeps the project small enough for beginners. You need three things: a Google Sheet with tidy tabs, a Flask app that fetches those tabs, and a few HTML pages that display the rows. There is no database and no login. The sheet is your database.

Scores flow one way: sheet, then Flask, then phones



data flow · 4 stages

Scorekeepers type into the sheet, Flask fetches and caches the data, and visitors only ever talk to Flask.

What to learn, in this order

Each person needs about 25 to 35 hours of learning in total, spread over the first two to three weeks. Learn each topic only until you pass its checkpoint, then start building. You will learn the rest by doing.

#	Topic	Why you need it	Good enough when you can…	Time
1	Git and GitHub	The whole team edits one codebase without overwriting each other	clone a repo, commit, push, pull, and make a branch	2–3 h
2	Python basics	Flask apps are Python	use variables, lists, dictionaries, loops, if statements and functions, and read an error message	8–10 h
3	Virtual environments and pip	Keeps Flask and other packages installed per project	create a venv, activate it, and run pip install flask	1 h
4	HTML and CSS basics	These make the pages people see	build a page with headings, a table, links and a simple style sheet	5–6 h
5	Flask basics	Routes, templates and static files are the core of the site	make an app with 3 routes, each showing a Jinja2 template	6–8 h
6	Reading CSV data in Python	Google Sheets can be fetched as CSV	download a CSV from a URL with requests and turn it into a list of dictionaries	2–3 h
7	Deploying a Flask app	So people can open it on their phones	put a hello-world Flask app online and open it from your phone	2–3 h

Later, and only if you have time: caching, the Google Sheets API with a service account (gspread library), and JavaScript for auto-refresh. You do not need a database, user accounts or JavaScript to finish this project.

Free places to learn: the official Flask Quickstart and Tutorial (flask.palletsprojects.com), the Python tutorial at docs.python.org, freeCodeCamp for HTML and CSS, and the GitHub Skills courses for Git. Split the topics so each person becomes the "go-to" on one of them and teaches the rest.

Setting up the Google Sheet

Design the sheet before writing any code, because Flask will depend on its exact layout. Use one tab per page, with column names in row 1 and one row per match or team.

Tab	Columns (row 1)
Football	Match, Time, Team A, Team B, Score A, Score B, Status
Volleyball	Match, Time, Team A, Team B, Sets A, Sets B, Status
Points	Team, Football Pts, Volleyball Pts, Other Pts, Total

Status is one of Upcoming, Live or Final. In the Points tab, let the sheet do the maths with a formula like =SUM(B2:D2) in the Total column, so Flask only displays and sorts.

Rules that prevent most bugs:

- No merged cells, no blank rows, and no extra notes above the header row.
- Spell team names identically in every tab.
- Keep a separate "Admin" tab for anything messy, and never publish it.
- Give edit access only to the scorekeepers.

How Flask gets the data. Start with the simplest route: in Google Sheets choose File, Share, Publish to web, pick one tab, choose the CSV format, and copy the link. Your Python code downloads that link and reads the rows. No Google account setup or secret keys are needed. The catch is that anyone with the link can read the data, which is fine for public sports day scores. If you later need it private, switch to the Google Sheets API with a service account and the gspread library.

Published sheets take a short while to update, so test how long the delay is early. Fetch each tab at most once every 30 to 60 seconds and keep the last result in memory, so the site stays fast and still shows data if Google is slow.

Build plan: six phases over about six weeks

Work backwards from your sports day date and stretch or squeeze the weeks to fit. Each phase ends with a "done when" check that the whole team can see working before moving on.

Phase 1: Set up and start learning (week 1)

- [ ] Create a GitHub repository and add everyone as collaborators
- [ ] Everyone installs Python and VS Code and clones the repo
- [ ] Each person creates a virtual environment and installs Flask
- [ ] Run a hello-world Flask app on every laptop
- [ ] Start topics 1 to 3 from the learning table

Done when every team member can run the same Flask app from the repo and push a small change.

Phase 2: Pages with fake data (weeks 2 to 3)

- [ ] Create the project structure below
- [ ] Make one shared base.html layout with a navigation bar
- [ ] Build three routes: /football, /volleyball and /points
- [ ] Put made-up results in Python lists and show them in HTML tables with Jinja2 loops
- [ ] Finish topics 4 and 5

Done when all three pages display fake data and look reasonable on a phone.

```
sports-day/
  app.py            # Flask routes
  sheets.py         # fetches and cleans data from Google Sheets
  templates/
    base.html
    football.html
    volleyball.html
    points.html
  static/
    style.css
  requirements.txt
```

Phase 3: Connect the Google Sheet (weeks 3 to 4)

- [ ] Build the sheet using the layout in the previous section and fill it with test data
- [ ] Publish each tab as CSV and save the links in one config place

- [ ] Write one function in sheets.py that takes a tab name and returns a list of dictionaries
- [ ] Replace the fake data in each route with that function
- [ ] Add a simple cache that refreshes every 30 to 60 seconds
- [ ] Finish topic 6

Done when you change a score in the sheet and the website shows the new score within about a minute.

Phase 4: Polish and make it robust (weeks 4 to 5)

- [ ] Sort the points table by Total, highest first
- [ ] Highlight Live matches and show Final results clearly
- [ ] Show a friendly message if the sheet cannot be reached, plus the last good data
- [ ] Handle empty cells and unexpected values without crashing
- [ ] Make a home page with the top three teams and any live matches

Done when a deliberately broken sheet (blank cells, a misspelled team) does not crash the site.

Phase 5: Deploy and rehearse (weeks 5 to 6)

- [ ] Finish topic 7 and put the site online
- [ ] Open it on several phones, on school Wi-Fi and on mobile data
- [ ] Run a mock sports day: scorekeepers enter 20 fake results while someone watches the site
- [ ] Write a one-page instruction sheet for scorekeepers
- [ ] Prepare a backup plan (see risks below)

Done when the mock day runs with no surprises and every scorekeeper knows exactly what to type where.

Phase 6: Sports day

- [ ] One teammate watches the site all day and fixes sheet mistakes quickly
- [ ] Share the link on posters, a QR code, or your school channels
- [ ] Note what broke or was missing for next year

Team roles, risks and tips

Split the work by role, and let people double up if the team is small. Rotate roles on small tasks so everyone learns a bit of everything.

Role	Owns	Main topics
Data lead	Sheet design, test data, scorekeeper instructions	Google Sheets, CSV
Backend	app.py, sheets.py, caching, error handling	Python, Flask, requests
Frontend	Templates and style.css, phone layout	HTML, CSS, Jinja2
Integrator	GitHub repo, reviewing changes, deployment	Git, deployment

Risks and fallbacks

- Scorekeepers mistype a team name: use dropdown lists (data validation) in the sheet for team names and Status.
- Wi-Fi or the host fails on the day: keep a laptop running the app locally, and as a last resort post the Points tab as a printed or photographed table.
- The free host is slow or blocks requests to Google: test this in Phase 5, not on sports day. Check the current limits of the host you pick, for example PythonAnywhere or Render.
- Too many features: finish football, volleyball and points first. Add other sports or auto-refresh only after the mock day passes.
- Everyone stuck on the same bug: set a rule that after 30 minutes stuck, you ask a teammate or a teacher, and when you ask, share the last line of the error message.

Habits that help beginners

- Build the smallest working version first, then improve it.
- Commit to Git often, with one feature per branch.
- Run the app after every small change, so you know which change broke it.
- Pair up: one person types, one reads and thinks.